EXPERIMENT 2: Mandatory Access Control (MAC) using AppArmor Works on Ubuntu VM.

1. AIM

To configure and test Mandatory Access Control (MAC) using AppArmor by creating a simple profile that restricts access to a folder and analysing denied operations.

2. THEORY

Mandatory Access Control (MAC)

MAC enforces system-level policies that override user permissions.

AppArmor Features:

- Profile-based program restrictions
- Prevents programs from reading specific files/folders
- Logs denied operations in /var/log/syslog

MAC > DAC:
Even if file permissions allow access, AppArmor can still block it.

3. PRE-SETUP STEPS

Install AppArmor Tools

Open Terminal:

```
sudo apt install apparmor apparmor-utils -y
```

- **sudo**: Executes the installation with administrative privileges required to modify system files and repositories.
- **apt install**: Uses the Advanced Package Tool (APT) to download and install software from the official Ubuntu repositories.
- **apparmor**: Installs the core service that runs in the Linux kernel to enforce security rules.
- **apparmor-utils**: Installs a suite of helper tools (like aa-status, aa-genprof, and aa-complain) that make it easier for you to create, manage, and troubleshoot profiles.
- **-y**: The "Assume Yes" flag. It automatically answers "yes" to any prompts during the installation, allowing the process to finish without waiting for you to press Enter.

Check status:

sudo aa-status

Breakdown of the Command

- **sudo**: Requires root privileges because the command queries the Linux kernel's security modules.
- **aa-status**: Short for **"AppArmor Status."** It provides a real-time report of the AppArmor engine and all loaded profiles.

4. OPERATIONAL STEPS

PART A — Create a Test Folder

Step 1: Create Folder

mkdir /home/testfolder

echo "secret information" > /home/testfolder/secret.txt

these commands establish the target environment. You are creating a specific file that contains "sensitive" data to test if AppArmor can successfully protect it.

Breakdown of the Commands

- **mkdir /home/testfolder**:
  - **Function**: Creates a new directory named testfolder directly under the root /home path.
  - **Purpose**: This provides a neutral location (outside of a specific user's home) to test system-wide security policies.
- **echo "secret information" > /home/testfolder/secret.txt**:
  - **echo**: A command that outputs the text provided to it.
  - **> (Redirection)**: This symbol takes the output of the echo command and writes it into a file. If the file doesn't exist, it creates it.
  - **/home/testfolder/secret.txt**: The specific file being created to hold the "secret" data.

Step 2: Ensure Everyone Can Read It

sudo chmod 777 /home/testfolder

Breakdown of the Command

- **sudo**: Required to change permissions on a directory located in a system path like /home/.
- **chmod**: The utility used to **CH**ange the **MOD**e (permissions) of a file or directory.
- **777**: The most permissive setting possible in Linux.
    - **7 (Owner)**: Read, Write, and Execute.
    - **7 (Group)**: Read, Write, and Execute.
    - **7 (Others)**: Read, Write, and Execute.
- **/home/testfolder**: The target directory created in the previous step.

PART B — Create an AppArmor Profile

Step 3: Create Profile File

sudo nano /etc/apparmor.d/usr.bin.cat-block

Paste:

```
# Block cat from reading the testfolder
/usr/bin/cat {
  deny /home/testfolder/** r,
}
```

you are transitioning from setting up the environment to defining the actual security policy. You are creating a "Profile" that tells the Linux kernel exactly how a specific application should behave.

Breakdown of the Configuration

- **sudo nano /etc/apparmor.d/usr.bin.cat-block**:
    - **/etc/apparmor.d/**: This is the standard directory where all AppArmor security profiles are stored.
    - **usr.bin.cat-block**: The filename convention usually reflects the path of the executable it regulates (in this case, /usr/bin/cat).
- **The Profile Syntax**:

- **/usr/bin/cat { ... }**: This defines the **attachment**. It tells the system that these rules apply *only* when the cat program is executed.
- **deny**: This is an explicit instruction to block access. In AppArmor, deny rules always take precedence over allow rules.
- **/home/testfolder/****: The double asterisk (**) is a globbing wildcard that represents the directory and all files/subdirectories within it.
- **r**: Specifies the **Read** permission.

PART C — Activate the Profile

Step 4: Load Profile

sudo apparmor_parser -r /etc/apparmor.d/usr.bin.cat-block

This is the moment the "rules" actually become "laws" that the system must obey.

Breakdown of the Command

- **sudo**: Requires root privileges because modifying kernel-level security modules is a restricted administrative task.
- **apparmor_parser**: The specialized utility that reads your text-based profile, checks it for errors, and converts it into a binary format that the kernel can understand.
- **-r (Replace)**: This flag tells the system to overwrite any existing version of this profile. It ensures that your latest changes to the cat-block file take effect immediately without needing a system reboot.
- **/etc/apparmor.d/usr.bin.cat-block**: The specific path to the profile file you created in Step 3.

Step 5: Test Access

Run:

cat /home/testfolder/secret.txt

Breakdown of the Test

- **The Command**: cat /home/testfolder/secret.txt
- **The Actor**: The cat binary (program).

- **The Target**: A file that has 777 (full access) permissions, which should technically be readable by anyone.

Why the result is "Permission Denied"

Even if you are the root user or the owner of the file, the command will fail. Here is the technical reason:

1. **Kernel Interception**: When you execute the command, the Linux kernel identifies that the program being used is cat. It immediately checks if there is an active AppArmor profile for /usr/bin/cat.
2. **Policy Enforcement**: The kernel finds your custom profile and sees the specific deny rule for the /home/testfolder/ path.
3. **Mandatory Override**: Because MAC is **Mandatory**, it acts as a higher authority than the file system. The kernel ignores the 777 permissions and blocks the cat program from opening the file before it can read even one byte of data.
4. **Error Message**: You receive a "Permission denied" or "Input/output error" message. This is not a standard user-permission error; it is a hard block from the security kernel.

You should see:

- Permission denied
- Input/output error
- Access blocked (even though DAC allows it)

PART D — View Logs

Step 6: Check Denied Operations

sudo grep apparmor /var/log/syslog

Breakdown of the Command

- **sudo**: Requires administrative privileges to read the system log files.
- **grep apparmor**: Searches for the specific word "apparmor" within the file.
- **/var/log/syslog**: The central location where Ubuntu and other Linux systems store general system logs, including security events.

What the Log Entry Means

When you run this, you will see a line similar to this: type=AVC msg=audit(...): apparmor="DENIED" operation="open" profile="/usr/bin/cat" name="/home/testfolder/secret.txt" pid=1234 comm="cat"

Here is how to interpret those "fingerprints":

- **apparmor="DENIED"**: Confirms that the security module was the reason the action failed.
- **operation="open"**: Shows that the cat program tried to open the file but was stopped.
- **profile="/usr/bin/cat"**: Identifies exactly which security profile triggered the block.
- **name="/home/testfolder/secret.txt"**: Confirms the specific target that the program was trying to access.
- **comm="cat"**: Confirms the command that was used.

You should see entries like:

DENIED r for /usr/bin/cat on /home/testfolder/secret.txt

5. EXPECTED OUTPUT

- The cat program is blocked from accessing /home/testfolder
- AppArmor logs the denied access
- Demonstrates MAC overriding DAC

6. CONCLUSION

In this experiment, we saw how MAC provides stronger security than DAC by enforcing system-wide policies using AppArmor. Even with full DAC permissions, MAC policies successfully blocked access and logged violations.